

01 — Docker (How One Java App Becomes a Container)

The problem Docker solves

"It works on my machine" — your app needs Java 21, a specific Postgres, specific ports.

Docker packs the app **with everything it needs** into one box, so it runs identically on your laptop, your teammate's laptop, and the cloud.

The 3 words

```
Dockerfile → docker build → IMAGE → docker run → CONTAINER
(recipe)                (frozen meal)        (meal being eaten)
```

This project's Dockerfile (every service has the same one)

```
FROM eclipse-temurin:21-jre-alpine # start from a tiny Linux with Java 21
COPY target/*.jar app.jar          # put your compiled Spring Boot jar in
ENTRYPOINT ["java","-jar","app.jar"] # when the container starts, run the jar
```

Three lines. That's the whole recipe.

docker-compose.yml — run ALL 19 containers with one command

Instead of typing `docker run` 19 times, `docker-compose.yml` lists everything:

6 services + 5 Postgres + Redis + Kafka + Kong + Floci. One command starts the lot:

```
docker-compose up --build -d
```

The 3 patterns inside compose (this is 90% of the file)

1. Containers find each other BY NAME, never by localhost:

```
SPRING_DATASOURCE_URL: jdbc:postgresql://postgres-user:5432/userdb
#                ^^^^^^^^^^^^^^^ container name = hostname
```

Inside Docker's network, `localhost` means "this container itself" — a classic beginner bug. `postgres-user` is the name of the DB container, and Docker's internal DNS resolves it. **2. Ports map outside:inside:**

```
ports:
  - "5433:5432"  # your laptop's 5433 → container's 5432
```

Five Postgres containers all listen on 5432 *inside*; they get different *outside* ports (5432-5436) so they don't clash on your machine. **3. Secrets come from `.env`, never hardcoded:**

```
POSTGRES_PASSWORD: ${USER_DB_PASS}  # value lives in .env (gitignored)
```

Volumes — where data survives

```
volumes:
  - postgres_user_data:/var/lib/postgresql/data
```

A **named volume** = a disk Docker manages. Kill the container, data stays. Without it, every restart = empty database.

Try it

```
docker-compose up -d      # start everything
docker ps                 # list running containers
docker logs user-service  # read one service's logs
docker-compose down       # stop (data kept)
docker-compose down -v    # stop AND wipe data
```

Next: **02-KUBERNETES.md** — what happens when you need to run these containers across many machines with automatic healing and scaling.